

第 1 章

OpenCV 入门

OpenCV 是一个开源的计算机视觉库，1999 年由英特尔的 Gary Bradski 启动。Bradski 在访学过程中注意到，在很多优秀大学的实验室中，都有非常完备的内部公开的计算机视觉接口。这些接口从一届学生传到另一届学生，对于刚入门的新人来说，使用这些接口比重复造轮子方便多了。这些接口可以让他们在之前的基础上更有效地开展工作。OpenCV 正是基于为计算机视觉提供通用接口这一目标而被策划的。

由于要使用计算机视觉库，用户对处理器（CPU）的要求提升了，他们希望购买更快的处理器，这无疑会增加英特尔的产品销量和收入。这也许就解释了为什么 OpenCV 是由硬件厂商而非软件厂商开发的。当然，随着 OpenCV 项目的开源，目前其已经得到了基金会的支持，很大一部分研究主力也转移到了英特尔之外，越来越多的用户为 OpenCV 做出了贡献。

OpenCV 库由 C 和 C++ 语言编写，涵盖计算机视觉各个领域内的 500 多个函数，可以在多种操作系统上运行。它旨在提供一个简洁而又高效的接口，从而帮助开发人员快速地构建视觉应用。

OpenCV 更像一个黑盒，让我们专注于视觉应用的开发，而不必过多关注基础图像处理的具体细节。就像 PhotoShop 一样，我们可以方便地使用它进行图像处理，我们只需要专注于图像处理本身，而不需要掌握复杂的图像处理算法的具体实现细节。

本章将介绍 OpenCV 的具体配置过程及基础使用方法。

1.1 如何使用

Python 的开发环境有很多种，在实际开发时我们可以根据需要进行选择一种适合自己的。在本书中，我们选择使用 Anaconda 作为开发环境。本节简单介绍如何配置环境，来实现在 Anaconda 下使用基于 Python 语言的 OpenCV 库。

1. Python 的配置

在本书中，我们使用的是 Python 3 版本。虽然 Python 2 和 Python 3 有很多相同之处，以至于 Python 2 的读者也可以使用本书，但还是要说明一下，本书直接面向的版本是 Python 3。

可以在 Python 的官网上（<https://www.python.org/downloads/>）下载 Python 3 的解释器。在该网页顶部已经指出了最新的版本，例如“Download Python 3.7.0”就表示当前（2018 年 10 月 9 日）的最新版本是 Python 3.7.0，单击“Download Python 3.7.0”，就会开始下载解释器软件。



如果想安装其他版本，可以向下拖动鼠标，你将看到一个滚动页面，其中显示了一个列表集，其中列出了不同的安装包，具体选择哪种安装包依赖于两个因素：

- 操作系统，例如 Windows、macOS、Linux 等。
- 处理器位数，例如 32 位或者 64 位。

根据自己的计算机配置，在列表集中选择对应的安装包下载。

下载完成后，按照提示步骤完成安装即可。

2. Anaconda的配置

可以在 Anaconda 的官网上 (<https://www.anaconda.com/download/>) 下载 Anaconda。在下载页顶部指出了当前的最新版本。

如果想安装其他版本，可以在下载页内根据实际情况选择。具体选择哪种安装包依赖于三个因素：

- Python 的版本，例如 Python 2.7 或者 Python 3.7 等。
- 操作系统，例如 Windows、macOS、Linux 等。
- 处理器位数，例如 32 位或者 64 位。

根据自己的环境配置，在下载页中选择对应的安装包下载。

下载完成后，按照提示步骤完成安装即可。

3. OpenCV的配置

可以从官网下载 OpenCV 的安装包，编译后使用；也可以直接使用第三方提供的预编译包安装。

在本书中，我们选择由 PyPI 提供的 OpenCV 安装包，可以在 <https://pypi.org/project/opencv-python/> 上面下载最新的基于 Python 的 OpenCV 库。同样，下载页顶部指出了当前的最新版本。

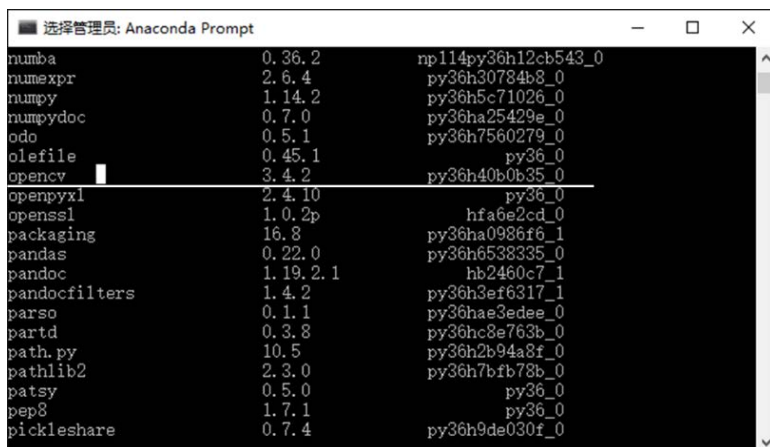
如果想安装其他版本，可以在 Download files 栏目内根据实际情况选择。具体选择哪种安装包依赖于三个因素：

- Python 的版本，例如 Python 2.7 或者 Python 3.7 等。
- 操作系统，例如 Windows、macOS、Linux 等。
- 处理器位数，例如 32 位或者 64 位。

完成下载后，在 Anaconda Prompt 内使用 `pip install` 完整路径文件名完成安装。例如，假设文件存储在 D:\anaconda\Lib 目录下面，则要使用的语句为：

```
>>>pip install D:\anaconda\Lib\opencv_python-3.4.3.18-cp37-cp37m-win_amd64.whl
```

安装完成后，可以在 Anaconda Prompt 内使用 `conda list` 语句查看安装是否成功。如果安装成功，就会显示安装成功的 OpenCV 库及对应的版本等信息。需要注意，不同安装包的名称及版本号可能略有差异。例如，图 1-1 中的包列表显示了系统内配置了 OpenCV 其他版本的情况，该图说明在当前系统内配置的是 OpenCV 3.4.2 版本。



Package Name	Version	Hash
numba	0.36.2	np114py36h12cb543_0
numexpr	2.6.4	py36h30784b8_0
numpy	1.14.2	py36h5c71026_0
numpydoc	0.7.0	py36ha25429e_0
odo	0.5.1	py36h7560279_0
olefile	0.45.1	py36_0
opencv	3.4.2	py36h40b0b35_0
openpyxl	2.4.10	py36_0
openssl	1.0.2p	hfabe2cd_0
packaging	16.8	py36ha0986f6_1
pandas	0.22.0	py36h6538335_0
pandoc	1.19.2.1	hb2460c7_1
pandocfilters	1.4.2	py36h3ef6317_1
parso	0.1.1	py36hae3edee_0
partd	0.3.8	py36hc8e763b_0
path.py	10.5	py36h2b94a8f_0
pathlib2	2.3.0	py36h7bfb78b_0
patsy	0.5.0	py36_0
pep8	1.7.1	py36_0
pickleshare	0.7.4	py36h9de030f_0

图 1-1 包列表

这里主要介绍了如何在 Windows 系统下对环境进行配置,如果需要配置其他操作系统下的环境,请参考官网的具体介绍。

1.2 图像处理基本操作

在图像处理过程中,读取图像、显示图像、保存图像是最基本的操作。本节将简单介绍这几项基本操作。

1.2.1 读取图像

OpenCV 提供了函数 `cv2.imread()` 来读取图像,该函数支持各种静态图像格式。该函数的语法格式为:

```
retval = cv2.imread( filename[, flags] )
```

式中:

- `retval` 是返回值,其值是读取到的图像。如果未读取到图像,则返回“None”。
- `filename` 表示要读取的图像的完整文件名。
- `flags` 是读取标记。该标记用来控制读取文件的类型,具体如表 1-1 所示。表 1-1 中的第一列参数与第三列数值是等价的。例如 `cv2.IMREAD_UNCHANGED=-1`,在设置参数时,既可以使用第一列的参数值,也可以采用第三列的数值。

表 1-1 flags 标记值

值	含义	数值
<code>cv2.IMREAD_UNCHANGED</code>	保持原格式不变	-1
<code>cv2.IMREAD_GRAYSCALE</code>	将图像调整为单通道的灰度图像	0
<code>cv2.IMREAD_COLOR</code>	将图像调整为 3 通道的 BGR 图像。该值是默认值	1
<code>cv2.IMREAD_ANYDEPTH</code>	当载入的图像深度为 16 位或者 32 位时,就返回其对应的深度图像;否则,将其转换为 8 位图像	2
<code>cv2.IMREAD_ANYCOLOR</code>	以任何可能的颜色格式读取图像	4



续表

值	含义	数值
cv2.IMREAD_LOAD_GDAL	使用 gdal 驱动程序加载图像	8
cv2.IMREAD_REDUCED_GRAYSCALE_2	将图像转换为单通道灰度图像，并将图像尺寸减小 1/2	
cv2.IMREAD_REDUCED_COLOR_2	将图像转换为 3 通道 BGR 彩色图像，并将图像尺寸减小 1/2	
cv2.IMREAD_REDUCED_GRAYSCALE_4	始终将图像转换为单通道灰度图像，并将图像尺寸减小为原来的 1/4	
cv2.IMREAD_REDUCED_COLOR_4	将图像转换为 3 通道 BGR 彩色图像，并将图像尺寸减小为原来的 1/4	
cv2.IMREAD_REDUCED_GRAYSCALE_8	将图像转换为单通道灰度图像，并将图像尺寸减小为原来的 1/8	
cv2.IMREAD_REDUCED_COLOR_8	将图像转换为 3 通道 BGR 彩色图像，并将图像尺寸减小为原来的 1/8	
cv2.IMREAD_IGNORE_ORIENTATION	不以 EXIF 的方向为标记旋转图像	

函数 cv2.imread()能够读取多种不同类型的图像，具体如表 1-2 所示。

表 1-2 cv2.imread()函数支持的图像格式

图像	扩展名
Windows 位图	*.bmp、*.dib
JPEG 文件	*.jpeg、*.jpg、*.jpe
JPEG 2000 文件	*.jp2
便携式网络图形（Portable Network Graphics，PNG）文件	*.png
WebP 文件	*.webp
便携式图像格式（Portable Image Format）	*.pbm、*.pgm、*.ppm、*.pxm、*.pnm
Sun（Sun rasters）格式	*.sr、*.ras
TIFF 文件	*.tiff、*.tif
OpenEXR 图像文件	*.exr
Radiance 格式高动态范围（High-Dynamic Range，HDR）成像图像	*.hdr、*.pic
GDAL 支持的栅格和矢量地理空间数据	Raster、Vector 两大类

例如，想要读取当前目录下文件名为 lena.bmp 的图像，并保持按照原有格式读入，则使用的语句为：

```
lena=cv2.imread("lena.bmp",-1)
```

需要注意，上述程序要想正确运行，首先需要导入 cv2 模块，大多数常用的 OpenCV 函数都在 cv2 模块内。与 cv2 模块所对应的 cv 模块代表传统版本的模块。这里的 cv2 模块并不代表该模块是专门针对 OpenCV 2 版本的，而是指该模块引入了一个改善的 API 接口。在 cv2 模块内部采用了面向对象的编程方式，而在 cv 模块内更多采用的是面向过程的编程方式。

本书中所使用的模块函数都是 cv2 模块函数，为了方便理解，在函数名前面加了“cv2.”。但是如果函数名出现在标题中，那么希望突出的是该函数本身，所以未加“cv2.”。

【例 1.1】使用 cv2.imread()函数读取一幅图像。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
```

```
print(lena)
```

上述程序首先会读取当前目录下的图像 `lena.bmp`，然后使用 `print` 语句打印读取的图像数据。运行上述程序后，会输出图像的部分像素值，如图 1-2 所示。

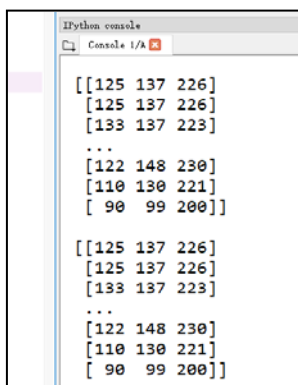


图 1-2 图像部分像素值

1.2.2 显示图像

OpenCV 提供了多个与显示有关的函数，下面对常用的几个进行简单介绍。

1. namedWindow函数

函数 `cv2.namedWindow()` 用来创建指定名称的窗口，其语法格式为：

```
None = cv2.namedWindow( winname )
```

式中，`winname` 是要创建的窗口的名称。

例如，下列语句会创建一个名为 `lesson` 的窗口：

```
cv2.namedWindow("lesson")
```

2. imshow函数

函数 `cv2.imshow()` 用来显示图像，其语法格式为：

```
None = cv2.imshow( winname, mat )
```

式中：

- `winname` 是窗口名称。
- `mat` 是要显示的图像。

【例 1.2】

在一个窗口内显示读取的图像。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.namedWindow("lesson")
cv2.imshow("lesson", lena )
```



在本程序中，首先通过 `cv2.imread()` 函数读取图像 `lena.bmp`，接下来通过 `cv2.namedWindow()` 函数创建一个名为 `lesson` 的窗口，最后通过 `cv2.imshow()` 函数在窗口 `lesson` 内显示图像 `lena.bmp`。

运行上述程序，得到的运行结果如图 1-3 所示。

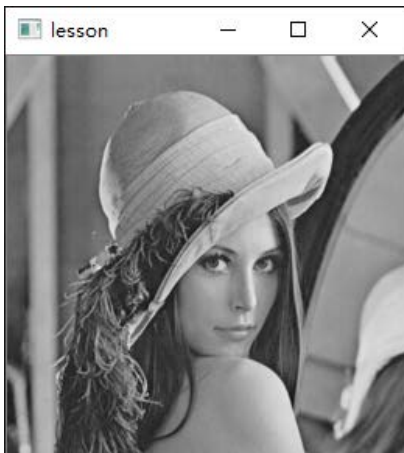


图 1-3 【例 1.2】程序的运行结果

在实际使用中，可以先通过函数 `cv2.namedWindow()` 来创建一个窗口，再让函数 `cv2.imshow()` 引用该窗口来显示图像。也可以不创建窗口，直接使用函数 `cv2.imshow()` 引用一个并不存在的窗口，并在其中显示指定图像，这样函数 `cv2.imshow()` 实际上会完成如下两步操作。

第 1 步：函数 `cv2.imshow()` 创建一个指定名称的新窗口。

第 2 步：函数 `cv2.imshow()` 将图像显示在刚创建的窗口内。

例如，在下面的语句中，函数 `cv2.imshow()` 完成了创建 `demo` 窗口和显示 `image` 图像的操作，该语句的功能与例 1.2 相同。

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.imshow("demo", lena )
```

在显示图像时，初学者最经常遇到的一个错误是 “`error: (-215:Assertion failed) size.width>0 && size.height>0 in function 'cv::imshow'`”，说明当前要显示的图像是空的（None），这通常是由于在读取文件时没有找到图像文件造成的。一般来说，没有找到要读取的图像文件，可能是因为文件名错误；如果确认要读取的图像的完整文件名（路径名和文件名）没有错，那么往往是工作路径配置错误造成的。

例如，当前程序所在路径为 `e:\lesson`，要读取当前路径下存在的图像 `image\lena.bmp`。那么：

- 如果设置的当前工作路径是 `e:\lesson`，程序就会读取当前工作路径下的文件 `image\lena.bmp`，该文件的完整文件名就是 `e:\lesson\image\lena.bmp`。正是我们想要读取的指定文件。
- 如果设置的当前工作路径不是 `e:\lesson`，例如，当前的工作路径是 `e:\python`，程序仍然会读取当前工作路径下的文件 `image\lena.bmp`（而不是读取当前程序所在路径下的

image\lena.bmp), 该文件的完整文件名就是 e:\python\image\lena.bmp。显然, 这样是无法读取到指定图像的。

为了避免上述错误, 可以在读取图像前判断图像文件是否存在, 并在显示图像前判断图像是否存在。

3. waitKey函数

函数 cv2.waitKey()用来等待按键, 当用户按下键盘后, 该语句会被执行, 并获取返回值。其语法格式为:

```
retval = cv2.waitKey( [delay] )
```

式中:

- `retval` 表示返回值。如果没有按键被按下, 则返回-1; 如果有按键被按下, 则返回该按键的 ASCII 码。
- `delay` 表示等待键盘触发的时间, 单位是 ms。当该值是负数或者零时, 表示无限等待。该值默认为 0。

在实际使用中, 可以通过函数 cv2.waitKey()获取按下的按键, 并针对不同的键做出不同的反应, 从而实现交互功能。例如, 如果按下 A 键, 则关闭窗口; 如果按下 B 键, 则生成一个窗口副本。

下面通过一个示例演示如何通过函数 cv2.waitKey()实现交互功能。

【例 1.3】 在一个窗口内显示图像, 并针对按下的不同按键做出不同的反应。

函数 cv2.waitKey()能够获取按键的 ASCII 码。例如, 如果该函数的返回值为 97, 表示按下了键盘上的字母 a 键。通过将返回值与 ASCII 码值进行比较, 就可以确定是否按下了某个特定的键。例如, 通过语句“返回值==97”就可以判断是否按下了字母 a 键。

Python 提供了函数 ord(), 用来获取字符的 ASCII 码值。因此, 在判断是否按下了某个特定的按键时, 可以先使用 ord()函数获取该特定字符的 ASCII 码值, 再将该值与 cv2.waitKey()函数的返回值进行比较, 从而确定是否按下了某个特定的键。这样, 在程序设计中就不需要 ASCII 值的直接参与了, 从而避免了使用 ASCII 进行比较可能带来的不便。例如, 要判断是否按下了字母 A 键, 可以直接使用“返回值==ord('A')”语句来完成。

根据题目要求及以上分析, 编写代码如下:

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.imshow("demo", lena )
key=cv2.waitKey()
if key==ord('A'):
    cv2.imshow("PressA", lena)
elif key==ord('B'):
    cv2.imshow("PressB", lena)
```

运行上述程序, 按下键盘上的 A 键或者 B 键, 会在一个新的窗口内显示图像 lena.bmp, 它们的不同之处在于:



- 如果按下的是键盘上的 A 键，则新出现的窗口名称为 PressA，如图 1-4 的左图所示。
- 如果按下的是键盘上的 B 键，则新出现的窗口名称为 PressB，如图 1-4 的右图所示。

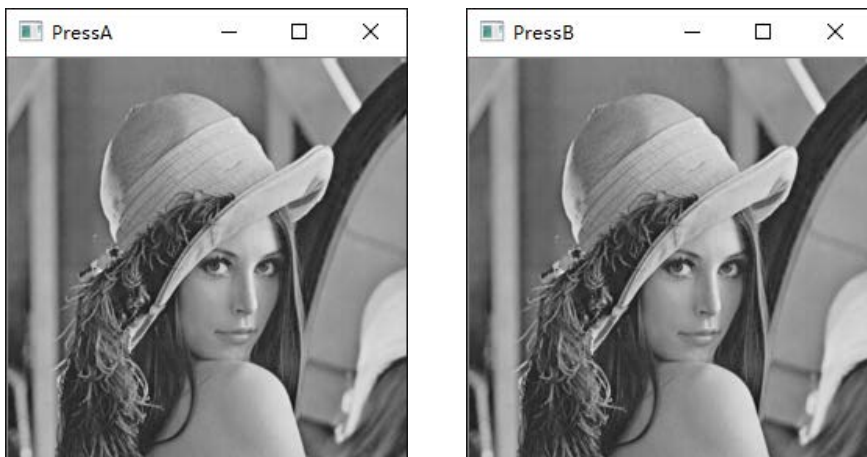


图 1-4 【例 1.3】程序的运行结果

从另外一个角度理解，该函数还能够让程序实现暂停功能。当程序运行到该语句时，会按照参数 `delay` 的设定等待特定时长。根据该值的不同，可能有不同的情况：

- 如果参数 `delay` 的值为 0，则程序会一直等待。直到有按下键盘按键的事件发生时，才会执行后续程序。
- 如果参数 `delay` 的值为一个正数，则在这段时间内，程序等待按下键盘按键。当有按下键盘按键的事件发生时，就继续执行后续程序语句；如果在 `delay` 参数所指定的时间内一直没有这样的事件发生，则超过等待时间后，继续执行后续的程序语句。

【例 1.4】 在一个窗口内显示图像，用函数 `cv2.waitKey()` 实现程序暂停，在按下键盘的按键后让程序继续运行。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.imshow("demo", lena )
key=cv2.waitKey()
if key!=-1:
    print("触发了按键")
```

运行上述程序，首先会在一个名为 `demo` 的窗口内显示 `lena.bmp` 图像。在未按下键盘上的按键时，程序没有新的状态出现；当按下键盘上的任意一个按键后，会在控制台打印“触发了按键”。

在本例中，由于 `cv2.waitKey()` 函数的参数值是默认值 0，所以在未按下键盘上的按键时，程序会一直处于暂停状态。当按下键盘上的任意一个按键时，程序中 `key=cv2.waitKey()` 下方的语句便得以执行，程序输出“触发了按键”。

4. destroyWindow函数

函数 `cv2.destroyAllWindows()` 用来释放（销毁）指定窗口，其语法格式为：

```
None = cv2.destroyAllWindows( winname )
```

其中，`winname` 是窗口的名称。

在实际使用中，该函数通常与函数 `cv2.waitKey()` 组合实现窗口的释放。

【例 1.5】 编写一个程序，演示如何使用函数 `cv2.destroyAllWindows()` 释放窗口。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.imshow("demo", lena )
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，首先会在一个名为 `demo` 的窗口内显示 `lena.bmp` 图像。在程序运行的过程中，当未按下键盘上的按键时，程序没有新的状态出现；当按下键盘上的任意一个按键后，窗口 `demo` 会被释放。

5. destroyAllWindows函数

函数 `cv2.destroyAllWindows()` 用来释放（销毁）所有窗口，其语法格式为：

```
None = cv2.destroyAllWindows( )
```

【例 1.6】 编写一个程序，演示如何使用函数 `cv2.destroyAllWindows()` 释放所有窗口。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
cv2.imshow("demo1", lena )
cv2.imshow("demo2", lena )
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会分别出现名称为 `demo1` 和 `demo2` 的窗口，在两个窗口中显示的都是 `lena.bmp` 图像。在未按下键盘上的按键时，程序没有新的状态出现；当按下键盘上的任意一个按键后，两个窗口都会被释放。

1.2.3 保存图像

OpenCV 提供了函数 `cv2.imwrite()`，用来保存图像，该函数的语法格式为：

```
retval = cv2.imwrite( filename, img[, params] )
```

式中：

- `retval` 是返回值。如果保存成功，则返回逻辑值真（True）；如果保存不成功，则返回逻辑值假（False）。
- `filename` 是要保存的目标文件的完整路径名，包含文件扩展名。



- `img` 是被保存图像的名称。
- `params` 是保存类型参数，是可选的。

【例 1.7】 编写一个程序，将读取的图像保存到当前目录下。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
r=cv2.imwrite("result.bmp",lena)
```

上述程序会先读取当前目录下的图像 `lena.bmp`，生成它的一个副本图像，然后将该图像以名称 `result.bmp` 存储到当前目录下。

1.3 OpenCV 贡献库

目前，OpenCV 库包含如下两部分。

- OpenCV 主库：即通常安装的 OpenCV 库，该库是成熟稳定的，由核心的 OpenCV 团队维护。
- OpenCV 贡献库：该扩展库的名称为 `opencv_contrib`，主要由社区开发和维护，其包含的视觉应用比 OpenCV 主库更全面。需要注意的是，OpenCV 贡献库中包含非 OpenCV 许可的部分，并且包含受专利保护的算法。因此，在使用该模块前需要特别注意。

OpenCV 贡献库中包含了非常多的扩展模块，举例如下。

- `bioinspired`：生物视觉模块。
- `datasets`：数据集读取模块。
- `dnn`：深度神经网络模块。
- `face`：人脸识别模块。
- `matlab`：MATLAB 接口模块。
- `stereo`：双目立体匹配模块。
- `text`：视觉文本匹配模块。
- `tracking`：基于视觉的目标跟踪模块。
- `ximgproc`：图像处理扩展模块。
- `xobjdetect`：增强 2D 目标检测模块。
- `xphoto`：计算摄影扩展模块。

可以通过以下两种方式使用贡献库：

- 下载 OpenCV 贡献库，使用 `cmake` 手动编译。
- 通过语句 `pip install opencv-contrib-python` 直接安装编译好的 OpenCV 贡献库。网页 <https://pypi.org/project/opencv-contrib-python/> 上提供了该方案的常见问题列表 FAQ (Frequently Asked Questions)，而且该 FAQ 是不断更新的。